

Partial Objects in the Calculus of Constructions

Philippe Audebaud
LaBRI - UFR Mathématiques & Informatique
351, Crs de la Libération
33405 Talence - France
e-mail : audebaud@geocub.greco-prog.fr

Abstract

Our purpose is to provide a typed framework for working with non terminating computations. The basic system is the Calculus of Constructions. It is extended using an original idea proposed by R.Constable and S.F.Smith and implemented in Nuprl. From the computational point of view we get an equivalent of Kleene's theorem for partial recursive functions over the integers within an index-free setting. A larger class of algebraic types is defined. Logical aspects need more examination. But we already give a syntactic way for dealing with partial and total objects, leading to the notion of generic proof.

1 Introduction

The Calculus of Constructions (*CC* for short) [8], [9] is a typed high-order functional calculus which provides a nice formalism for constructive proofs in natural deduction style. It can also be seen as a high-level functional programming language.

Since $F\omega$ is embeddable in it, we already know that any function over integers is definable on $Nat \equiv (C : Prop)C \rightarrow (C \rightarrow C) \rightarrow C$ if it is provably total in higher-order arithmetic [8].

However Church's representation of natural numbers is not satisfactory. Computing the predecessor of $\bar{n}$ requires n steps of reduction. The principle of induction is neither provable nor realizable [14]. The representation of recursive inductive types [15] seems to be very inefficient from the computational point of view. As pointed in that paper we miss the fact that convertibility rule should be an internal conversion.

On the other hand we could expect to find, in a typed framework, the classical results on recursive functions. In fact, dealing with typed terms should lead to a clearer framework. Moreover we agree with [6] when they emphasize the necessity of pointing out the minimal apparatus thus required.

This work is initially inspired from two papers of R.Constable and S.F.Smith [5], [6]. The underlying

idea was the following: with each type T comes a bar type $\bar{T}$ which may contain divergent computations. Fixpoints are allowed in $\bar{T}$ only (otherwise it is clear that the system is inconsistent). Convergent terms may then be typed in T by coercion.

In *Nuprl* propositions and types are just the same thing (Howard's isomorphism) and the interpretation is of computational nature: computation is done on untyped λ -terms. Thus convergence of computations is easily representable in the meta-theory. All the trouble begins when we want to type these convergent untyped expressions. In [16] Smith exhibits a restricted universe V of "admissibles types" in which fixpoints computations are allowed. Dependent sums and higher universes are essentially excluded from V .

In *CC* each term comes with the very proof of its correctness (through typing information). And the only constructor is that of dependent product. Thus none of the previous difficulties remains. The main problem we meet concerns coercion. In *CC* this seems to be a bad way to type terms since they carry too much typing information. Semantics reveals the deep difficulty in interpreting bar type variables.

We briefly point out our main results: we get a conservative extension of *CC* in which strong normalisation still holds for β -reduction. So consistency is preserved. The type *Nat* is given by a fixpoint. Internalization of the primitive recursion scheme succeeds this way. One can notice that we recover the representation proposed by Parigot-Krivine [13]. All the Peano's axioms are provable. All the partial recursive functions are definable (Kleene). The results obtained over the type *Nat* can be generalized to a large class of algebraic types.

2 CC^+ : an extension of the Calculus of Constructions

2.1 Overview

2.1.1 Terms The set of terms of CC^+ is Λ^+ . It is the least set of terms built from the kind constants

$Prop$, $\overline{Prop}$, $Type$ and $\overline{Type}$, a denumerable set V of variables and stable through the following constructs

application	$(M\ N)$
abstraction	$[x : M]N$
fixpoint	$\langle x : M \rangle N$
product	$(x : M).N$

where $M, N \in \Lambda^+$ and $x \in V$.

2.1.2 Reductions There are two notions of reduction:

$$\begin{aligned} \beta \quad & ([x : M]N\ L) \longrightarrow_{\beta} N[x/L] \\ \beta fix \quad & \langle x : M \rangle N \longrightarrow_{\beta fix} ([x : M]N\ \langle x : M \rangle N) \end{aligned}$$

where $M, N, L \in \Lambda^+$ and $x \in V$. We let $\rightarrow_+$ be the reflexive and transitive closure of $\beta \cup \beta fix$ and $=_+$ the congruence generated by it.

Our choice for βfix comes from the fact that if ω is a fixpoint for the function f then we do have $\omega = f(\omega)$. Choosing βfix as $\langle x : M \rangle N \longrightarrow_{\beta fix} N[x/\langle x : M \rangle N]$ would have hidden the fact that, in the Calculus of Constructions, all the fixpoints we may construct are actually fixpoints of functional terms. Moreover it would have led to a bad reduction behavior. For, dealing with fixpoint computations involves that we must take *head* reductions as meaningful.

2.1.3 Inference rules As in *CC* there are three kinds of rules: for valid contexts, for typing judgments and for equality judgments. They are shown in figure 1, except equality rules which call for no special remark. In fact we add few rules to *CC* ones. An introduction rule $\Delta \vdash \overline{Prop} \in \overline{Type}$ for the kind of “partial propositions” and, essentially, another one which allows fixpoints to be built.

Note carefully that all fixpoints have to be typed within a bar kind: $\overline{Prop}$ or $\overline{Type}$. Those introduced within $\overline{Prop}$ are just (partial) proofs (see 2.2.3). No restriction is required. As a counterpart we need some restriction when introducing propositional scheme via fixpoint in order to keep the system strongly normalizing through β -reductions.

$Pos_x(\cdot)$ stands for the usual predicate of positivity. It is introduced together with a predicate of negativity denoted by $Neg_x(\cdot)$. The two of them are defined in the usual way on product types. We have to extend these definitions to all the terms of Λ^+ . If Π_x stands for any of the two predicates we require that $\Pi_x([y : M_1]M_2)$ and $\Pi_x((M_2\ M_1))$ if $\Pi_x(M_2)$ and $x \notin FV(M_1)$ are both satisfied. Hence $\Pi_x(\langle y : M_1 \rangle M_2)$ is satisfied if $x \notin FV(\langle y : M_1 \rangle M_2)$. This is restrictive but necessary as shown by the example of $([y : \overline{Prop}](y \rightarrow x)\ x) \in \overline{Prop}$. And, since a

• empty

$$\overline{\square} \text{ is valid}$$

• var-introduction

$$\frac{\Delta \vdash M \in \mathcal{K} \quad x \in V \quad x \notin \Delta}{\Delta, x : M \text{ is valid}}$$

• hypothese

$$\frac{\Delta \vdash x \in V(\Delta)}{\Delta \vdash x \in \Delta(x)}$$

• Prop-introduction

$$\frac{\Delta \vdash}{\Delta \vdash Prop \in Type}$$

• $\overline{Prop}$ -introduction

$$\frac{\Delta \vdash}{\Delta \vdash \overline{Prop} \in \overline{Type}}$$

• prod-introduction

$$\frac{\Delta, x : P \vdash M \in \mathcal{S}}{\Delta \vdash (x : P)M \in \mathcal{S}}$$

• abs-introduction

$$\frac{\Delta, x : P \vdash M \in \mathcal{N}}{\Delta \vdash [x : P]M \in (x : P)N}$$

• appl

$$\frac{\Delta \vdash M \in (x : P)N \quad \Delta \vdash R \in Q \quad \Delta \vdash N = Q}{\Delta \vdash (M\ R) \in N[x/R]}$$

• fix-introduction-Prop

$$\frac{\Delta \vdash P \in \overline{Prop} \quad \Delta, x : P \vdash M \in P}{\Delta \vdash \langle x : P \rangle M \in P}$$

• fix-introduction-Type

$$\frac{\Delta \vdash P \in \overline{Type} \quad \Delta, x : P \vdash M \in P \quad s.t. Pos_x(M)}{\Delta \vdash \langle x : P \rangle M \in P}$$

• type-conversion

$$\frac{\Delta \vdash M \in N \quad \Delta \vdash N = N'}{\Delta \vdash M \in N'}$$

Figure 1: Rules of inference for CC^+

well-founded definition of these predicates is required *before* the proof of strong normalisation, we cannot allow us to consider only terms with no β -redexes.

2.2 Syntax for a well-formed term

In the following lines $\mathcal{J}_\Delta$ denotes the set of the judgments valid in the context Δ and $\mathcal{J}$ the collection of all judgments.

2.2.1 Definition Let E be any (non empty) set. A map $\varphi : \mathcal{J} \longrightarrow E$ is an **invariant** if

$$(\forall \Delta)(\forall M, N \in \mathcal{J}_\Delta) \Delta \vdash M = N \implies \varphi(M) =_E \varphi(N)$$

In the Calculus of Constructions the notions of *proof*, *proposition*, *propositional type* and *propositional scheme* are of great interest since they are invariant through type conversion. Many invariants have been already introduced in CC . Some of them explicitly: *degree*, *complexity*. . . Others are implicit. For example the **kind** of a term is a key invariant when defining the (non) informative terms [14]. Only one invariant is defined here. It will give us an easy way to check when we are allowed to construct a proposition (more generally a propositional scheme) through the fix-introduction rule.

2.2.2 Definition (i) Let $\Delta \vdash M \in N$ be a valid judgment. The **kind** of M , noted $\mathcal{S}(M)$, is N if N is a kind, $\mathcal{S}(N)$ otherwise.

(ii) A term of kind *Prop* or *Type* is a **total term**; otherwise it is a **partial one**.

2.2.3 Proposition Any well-formed term M is of one of the following forms (modulo α -conversion):

$$M \equiv [\overline{x : \vec{P}}](\overline{y : \vec{Q}})(M' \ \vec{R})$$

with

$$M' \equiv \begin{cases} ([z : T]M_1 \ M_2) \\ \langle z : T \rangle M_1 \\ \text{constant or variable} \end{cases}$$

and

$$\mathcal{S}(M) = \mathcal{S}(M') = \mathcal{S}(M_1)$$

Where $\overline{x : \vec{M}}$ is for $x_1 : M_1 \dots x_k : M_k$, $k \geq 0$.

2.3 Basic results

We refer to [9] for all the basic results on the original calculus CC . We point out

2.3.1 Proposition $\beta \cup \beta\text{fix}$ is Church-Rosser (CR).

2.3.2 Theorem CC^+ is a conservative, hence consistent, extension of CC .

PROOF CC is clearly embeddable in CC^+ . Moreover, given a total term M , we can extract a term $\mathcal{E}(M)$ living in CC in the same way as in [9], [14]. $\square$

2.3.3 Warning ! Any partial proposition $P \in \overline{Prop}$ is provable since $\langle x : P \rangle x$ is always a valid proof for P .

3 Uniform properties on CC^+

We know about many properties like CR, strong normalisation for $\beta \cup \beta\text{fix}$ ($\text{SN}_{\beta \cup \beta\text{fix}}$), or β (SN_β), finiteness of developments (FD!), standardization (ST) which concern syntactically or provably well-formed terms. We call them **uniform** properties. The scheme of Girard's proof for SN_β in the system F stands out as THE scheme for reasoning in this kind of structure in the same way as the principle of induction for natural numbers. Actually this theorem gives us a sufficient condition for a property to be valid for all provably well-formed terms. The following is an extension of the proof given by Coquand for SN_β in CC . Variations occur on the form, inspired by Breazu-Tannen and Gallier papers [4], [10]. As they did, we only intent to give a clearer insight of the proof and of the way properties on terms can be proved.

3.1 Saturated sets of terms

3.1.1 Definition Let P be a property on terms of CC^+ , M a well-formed type and C a set of terms of type M having the property P .

(i) C is a **saturated set** if the following conditions hold

(Upper-set) $M \in C$ and $M \twoheadrightarrow_+ N \implies N \in C$

(Induction) M simple and $((\forall N) M \twoheadrightarrow_+ N \implies N \in C) \implies M \in C$

(ii) $\text{Sat}(M)$ is the set of all saturated sets of type M .

3.1.2 Fact $(\text{Sat}(M), \subseteq)$ is a consistently complete cpo.

3.2 Girard's theorem

3.2.1 Theorem (cf. [4]) Let P be a property of terms. If

(i) for any term M and constant c such that $(M \ c)$ has property $P \implies M$ has property P ,

(ii) for any type M the set formed by all terms of type M having property P is saturated.

Then any provably well-formed term has property P .

The proof requires many steps and definitions fully given elsewhere ([2]). We give a sketch of it below, following [7].

3.3 Full premodel

$Prop$, $\overline{Prop}$, $Type$, $\overline{Type}$ are special constants in Λ^+ . Let $\mathcal{C}$ be a fixed set of constants. Then $\mathcal{I} = \mathcal{C} \cup \mathcal{V}$ is defined as the set of **identifiers** of Λ^+ . A **premodel** is a well-formed environment Δ such that $V(\Delta) \subseteq \mathcal{C}$. Given a premodel Δ_0 a list Δ is a **relative environment** if $\Delta_0 \cup \Delta$ is a well-formed environment. The following result is easy:

3.3.1 Proposition There exists a premodel $\Delta^\vee$, called a **full premodel** such that, for any type T , well-formed in this premodel, there exists a constant of type T .

In the next lines we place ourselves in the full premodel $\Delta^\vee$.

3.3.2 Definition Let Δ a relative environment. A map φ of domain $V(\Delta)$ to the set of provably well-formed terms in $\Delta^\vee$ is a **closed substitution** if $\varphi|_{\Delta^\vee} = id_{\Delta^\vee}$ and

$$(\forall x \in V(\Delta)) \Delta^\vee \vdash \varphi(x) \in \varphi(\Delta(x)).$$

3.4 Partial applicative structure

Let M be a well-formed term. The set of **values** for M , $v(M)$, contains all pairs $\langle N, \pi \rangle$ such that $N =_+ M$ and π is defined by case

$$M \begin{cases} \text{a proof} & \pi \equiv \kappa \text{ (arbitrary fixed object)} \\ \text{a type} & \pi \subseteq \langle M \rangle \text{ such that } \text{tr } \pi \subseteq \text{Sat}(M) \\ \text{a scheme} & \pi \text{ is an application such that} \end{cases}$$

if $M \in (x : P)Q$, π has domain $\langle P \rangle$ and satisfies

$$(\forall \alpha \equiv \langle N, \rho \rangle) \pi(\alpha) \in v((M \ N))$$

where $\langle M \rangle \equiv \cup_{P \in M} v(P)$ and $\text{tr } \pi \equiv \{T \mid \langle T, \alpha \rangle \in \pi\}$ when M is a type. In that case $\langle T, \pi \rangle \in \langle M, \rho \rangle$ abbreviates $T \in \text{tr } \rho$. We can check that $v(M) = v(N)$ if $M =_+ N$.

When it makes sense we write $\langle M, \alpha \rangle \cdot \langle N, \beta \rangle$ for the value $\langle (M \ N), \alpha(\beta) \rangle$ and we fix $\kappa(\beta) \equiv \kappa$.

For $\alpha, \beta \in v(M)$ the previous definition allows us to introduce an order on the values in a very simple way: $\langle M, \pi \rangle < \langle N, \rho \rangle$ iff $M =_+ N$ and nothing else for a proof, $\text{tr } \pi \subseteq \text{tr } \rho$ when M is a type, by induction in the case of schema - using some invariant defined by T.Coquand as a complexity measure.

The partial applicative structure in which we will interpret the terms is the space of values for well-formed terms in the full premodel $\Delta^\vee$ equipped with the partial operation $\cdot$. We denote it by $(\mathfrak{M}, \cdot)$.

The cpo structure on $\text{Sat}(M)$ ensures the existence of a least value $\inf(M)$ w.r.t. $<$ in each $v(M)$. And

hypothesis (ii) from Girard's theorem gives rise to a greatest value $\sup(M)$. These two values are invariant through conversion of M .

Finally, if ω is a fixpoint and $\Phi : v(\omega) \rightarrow v(\omega)$ a monotonic map, we denote by $\text{Fix}(\Phi)$ its least fixed point, the existence of which is provided by the previous paragraph.

3.4.1 Definition Let Δ a relative environment and φ a closed substitution on Δ . A **valuation** on Δ is a map $\rho : V(\Delta) \rightarrow \mathfrak{M}$ such that

$$(\forall x \in V(\Delta)) \rho(x) \in v(\varphi(x)).$$

The order $<$ extends easily to valuations.

PROOF (sketch of Girard's theorem) Let Δ a relative environment and ρ a valuation on Δ . To each term M , well-formed in Δ , we associate a special value $\llbracket M \rrbracket_\rho$ in $\mathfrak{M}$ shown in figure 2. Notice that when M is a proof, then so is $\varphi(M)$.

The next steps are the usual ones

- (i) $(\forall M) \llbracket M \rrbracket_\rho \in v(\varphi(M))$
- (ii) $M =_+ M' \implies \llbracket M \rrbracket_\rho = \llbracket M' \rrbracket_\rho$

This interpretation also satisfies

$$\begin{aligned} \text{Pos}_X(M), \rho < \rho' &\implies \llbracket M \rrbracket_\rho < \llbracket M \rrbracket_{\rho'} \\ \text{Neg}_X(M), \rho < \rho' &\implies \llbracket M \rrbracket_\rho > \llbracket M \rrbracket_{\rho'} \end{aligned}$$

We finally have to check that the interpretation is correct. That is to say

$$(\forall x \in V(\Delta)) \llbracket x \rrbracket_\rho \in \llbracket \Delta(x) \rrbracket_\rho$$

implies

$$(\forall \Delta \vdash M \in N) \llbracket M \rrbracket_\rho \in \llbracket N \rrbracket_\rho$$

□

3.4.2 Corollary Proprieties SN_β , CR, FD! and ST are valid for the set of provably well-formed terms of CC^+ .

PROOF From 3.2.1 we have to check that each of these properties satisfies the hypotheses of the theorem. The only precision required is for FD!. We extend CC^+ by indexing β fix-redexes. However the interpretation $\llbracket \cdot \rrbracket_\rho$ extends naturally. □

Notice that this theorem gives us another proof of consistency for the Extended Calculus of Constructions CC^+ .

4 Definability of partial recursive functions

The definability of partial recursive functions (p.r. functions for short) in the pure λ -calculus is well-known (Kleene's theorem). The proof given by H.Barendregt [3] depends essentially on two points: (1) the existence of a representation of natural numbers as “sticks” (or lists of booleans) and (2) the property for that collection of λ -terms of being *uniformly solvable*. The fixpoint constructor allows us to satisfy these two requirements within the recursive inductive types framework proposed by Ch.Paulin-Mohring [15]. We have first to deal with the two points above.

4.1 Terms Almost in Normal Form

The following notion is the best generalisation of normal form we can define when dealing with β fix reductions.

4.1.1 Definition (i) The set ANF of terms **almost in normal form** is the least class of well-formed terms containing the kinds (ie the constants), the set $\mathcal{V}$ of variables and closed by

$$\begin{aligned} M \text{ ANF} &\implies \{x : P\}M \text{ ANF} \\ N_1 \dots N_k \text{ ANF} &\implies (x N_1 \dots N_k) \text{ ANF} \end{aligned}$$

where $\{\cdot\}$ is for $(\cdot)$ or $[\cdot]$, $P \in \Lambda^+$ being any type and $x \in \mathcal{V} \cup \mathcal{C}$.

(ii) Let M and N be ANF term. We define $M \doteq N$ by

- $\{x : P\}M \doteq \{x : Q\}N$ iff $M \doteq N$ and $P =_+ Q$
- $(x M_1 \dots M_k) \doteq (y N_1 \dots N_l)$ iff $k = l$, $x \equiv y$ and $(\forall i) M_i \doteq N_i$.

(iii) A term M has an **almost normal form** if there exists N ANF such that $N =_+ M$.

4.1.2 Facts (i) Any propositional type is an ANF.

(ii) $M \text{ ANF} \implies M \text{ HNF}$.

4.2 The type “Natural Numbers”

In the introduction we pointed out the limits of the usual polymorphic type

$$\text{Cnat} \equiv (C : \text{Prop})C \rightarrow (C \rightarrow C) \rightarrow C$$

in which the integers are built in as functional iterators (Church's numerals).

In fact other numerals systems exist. We propose the following definition:

4.2.1 Definition (i) the type “natural numbers” is given by

$$\text{Nat} \equiv \langle X : \overline{\text{Prop}} \rangle (C : \overline{\text{Prop}})C \rightarrow (X \rightarrow C) \rightarrow C$$

(ii) we let

$$\begin{aligned} \underline{0} &\equiv [C : \overline{\text{Prop}}][Z : C][S : \text{Nat} \rightarrow C]Z \\ \underline{\text{succ}} &\equiv [n : \text{Nat}][C, Z, S](S \ n) \\ \underline{\text{pred}} &\equiv [n : \text{Nat}](n \ \text{Nat} \ \underline{0} \ \mathbb{1}_{\text{Nat}}) \end{aligned}$$

where $\mathbb{1}_S$ stands for the combinator $[x : S]x$ of type $S \rightarrow S$.

For $n \in \mathbb{N}$ we will shorten $\underline{n-1} \equiv (\underline{\text{pred}} \ n)$ and $\underline{n+1} \equiv (\underline{\text{succ}} \ n)$.

4.2.2 Fact $(\forall n \in \text{Nat}) (\underline{\text{pred}} (\underline{\text{succ}} \ n)) \rightarrow_h n$

This is a key result when proving Peano's axioms for example. See problems related to inductive and recursive types within CC in [14] and [15]. It is worth noting that this simple property together with the type-conversion rule give us the only possible way (?) for introducing logical predicates on partial propositions, as far as no recursive definition such “if $x=0$ then A else B”, where A, B are propositions, is allowed in CC^+ . To be compared with predicate Z in [8].

We give a (non trivial) proof of $\neg(\underline{x+1} = 0)$ as an illustration of this fact in figure 3.

The construction of the predecessor reveals the essential property of a terms of type Nat : they are defined by case. Thus we *know*, as regard computations, whether $\underline{n}$ is *zero* or a *successor*. We will be more precise in the proof of Kleene's theorem (cf. 4.4.2).

Now we have

4.2.3 Proposition Nat is **uniformly solvable** : given any partial proposition S , there exist terms $M_1, \dots, M_K$ such that

$$(\forall n \in \mathbb{N}) (\underline{n} \ M_1 \dots M_K) \rightarrow_+ \mathbb{1}_S$$

PROOF (sketch) Let $S \in \overline{\text{Prop}}$ and $n \in \text{Nat}$. Then $(n \ S \rightarrow S \ \mathbb{1}_S \ [p] \ \mathbb{1}_S)$ works. From now on we let $\delta_S \equiv [n : \text{Nat}](n \ S \rightarrow S \ \mathbb{1}_S \ [p] \ \mathbb{1}_S)$. $\square$

In fact we have a built-in construct for *call-by-value* computations. For example, the term $\text{seq}(a, x.b)$ defined in [6] (which is unsolvable when a is unsolvable and returns $b[x/a]$ otherwise) is given by $(\delta_{\text{Nat}} \ a)([x : \text{Nat}]b \ a)$.

More generally

4.2.4 Definition Let $F : \text{Nat}^p \rightarrow \text{Nat}$ ($p \geq 1$) and $\vec{x} \in \text{Nat}^p$. Then

$$[\vec{x}](\delta_{\text{Nat}} x_1) \cdots (\delta_{\text{Nat}} x_p)(F \vec{x})$$

will be shortened $[\vec{x}]_{\downarrow}(F \vec{x})$ or $[\vec{x}](F \vec{x})_{\downarrow}$

4.3 Standard elements

The next proposition strongly motivates the consideration of the numerals which are either ANF or unsolvable.

4.3.1 Proposition Let t a closed term of type Nat . Then

$$t \text{ has an ANF} \iff (\exists n \in \mathbb{N}) t \doteq \underline{n}$$

Thus terms $\underline{n}$, where $n \in \mathbb{N}$, may be considered as unique to some extent. We are now able to give a precise sense, in our typed setting, to the way functions are usually considered to be *partial* ones in the common mathematical practice.

We restrict ourselves to closed terms of type $\text{Nat}^p \rightarrow \text{Nat}$, $p \geq 0$. $\text{Nat}^0 \rightarrow \text{Nat}$ is identified with Nat .

4.3.2 Definition (i) Let $M \in \text{Nat}^p \rightarrow \text{Nat}$ (a closed term). M is **standard** iff it is either unsolvable, denoted $M \uparrow$, or if it satisfies, for all $\vec{x} \in \text{Nat}^p$

- $(\exists i) x_i \uparrow \implies (M \vec{x}) \uparrow$
- $(M \vec{x})_{\downarrow} \implies (M \vec{x})$ has an ANF.

(ii) We write $\overline{\text{Nat}^p \rightarrow \text{Nat}}$ for the set of standard terms of type $\text{Nat}^p \rightarrow \text{Nat}$.

One may notice that $\overline{\text{Nat}} \equiv \{\uparrow\} \cup \{\underline{n} \mid n \in \mathbb{N}\}$ has the same meaning as the corresponding bar type in [6].

4.4 Partial recursive functions

4.4.1 Definition A partial numeric function $\varphi : \mathbb{N}^p \rightarrow \mathbb{N}$ is **definable** if there exists $F : \overline{\text{Nat}^p \rightarrow \text{Nat}}$ (thus standard) such that

$$(\forall \vec{x}) \begin{cases} (F \vec{x}) \doteq \varphi(\vec{x}) & \text{if } \varphi(\vec{x})_{\downarrow} \\ (F \vec{x}) \uparrow & \text{otherwise} \end{cases}$$

We let $\varphi \rightsquigarrow F$ in that case.

4.4.2 Theorem All partial recursive functions are definable.

PROOF Owing to the previous lines we can follow exactly Barendregt's proof for Kleene's theorem (see [3], § 6.3 and 8.4). As an example of the way we type new

terms in the extension we will fill in the details for the most interesting cases.

• **composition** Let $\psi_i \rightsquigarrow G_i$ ($1 \leq i \leq r$) and $\chi \rightsquigarrow H$. Then $\varphi = \chi(\psi_1, \dots, \psi_r)$ is definable by

$$[\vec{x} : \overline{\text{Nat}}]_{\downarrow}(H (G_1 \vec{x})_{\downarrow} \cdots (G_r \vec{x})_{\downarrow})$$

• **primitive recursion** Let φ, ψ and χ such that

$$\begin{aligned} \varphi(0, \vec{x}) &= \chi(\vec{x}) \\ \varphi(n+1, \vec{x}) &= \psi(\varphi(n, \vec{x}), n, \vec{x}) \end{aligned}$$

If $\chi \rightsquigarrow H$ and $\psi \rightsquigarrow G$ then φ is definable by

$$[n : \text{Nat}][\vec{x} : \overline{\text{Nat}}]_{\downarrow}(n \text{ Nat } (G \vec{x}) [p](H (F p \vec{x})_{\downarrow} p \vec{x}))$$

So we see that this scheme of recursion is given by the very structure of the terms in Nat .

• **μ -recursion** For $\chi \in \text{Nat}^{p+1} \rightarrow \text{Nat}$ we let φ be defined as " $\varphi(\vec{x})$ is the least $m \in \mathbb{N}$ s.t. $\chi(m, \vec{x}) = 0$ if it exists, undefined otherwise". We note

$$\varphi \stackrel{\text{def}}{=} \lambda \vec{x}. \mu m [\chi(m, \vec{x}) = 0].$$

Then, if $\chi \rightsquigarrow G$ we get $\varphi \rightsquigarrow [\vec{x} : \overline{\text{Nat}}]_{\downarrow}(H \vec{x} \underline{0})$ where

$$H = [\vec{x} : \overline{\text{Nat}}]_{\downarrow}(((G \vec{x} n) \text{ Nat } n [p](H \vec{x} \underline{n+1})))$$

□

4.5 Universal enumerative functions

We are now concerned with universal enumerative functions. With no loss of generality we restrict ourselves to the enumeration of p.r. functions with one argument. Through de Bruijn notation and extraction of a pure λ -term [9] we get the existence of a Gödel numbering $\ulcorner \cdot \urcorner$ for p.r. functions such that

4.5.1 Proposition (i) there exists two standard terms **code** and **decode** of type $\text{Nat} \rightarrow \text{Nat}$ with $(\text{code } \underline{n}) =_+ \ulcorner \underline{n} \urcorner$ each of the two being inverse of the other.

(ii) there exists a standard function **eval** of type $\text{Nat}^2 \rightarrow \text{Nat}$ such that for all M of type $\text{Nat} \rightarrow \text{Nat}$ and $n \in \mathbb{N}$, if $(M \underline{n})_{\downarrow}$ then

$$(\text{eval } \ulcorner M \urcorner (\text{code } \underline{n})) \doteq \ulcorner (M \underline{n}) \urcorner$$

PROOF To be complete the proof would require a great amount of place. Let us sketch some steps.

• Since any term M we used is closed and of type $\text{Nat}^p \rightarrow \text{Nat}$ we may erase any typing information (actually it is well typed, a priori). Thus we

get an extended untyped λ -term $\mathfrak{R}(M)$. For example $\mathfrak{R}((x : \text{Nat}^p \rightarrow \text{Nat})M_1)$ is $\mu x. \mathfrak{R}(M_1)$.

- $\ulcorner M \urcorner$ will be the Gödel number computed on the abstract representation $\mathfrak{R}'(M)$ of $\mathfrak{R}(M)$ as originated by de Bruijn. An example:

$$\mathfrak{R}(\underline{n+1}) \equiv \lambda Z. \lambda S. (S \mathfrak{R}(\underline{n}))$$

and

$$\mathfrak{R}'(\underline{n+1}) \equiv \lambda. \lambda. (1 \mathfrak{R}'(\underline{n}))$$

(since our terms are closed no lift operation is required here). Now (code $\underline{n+1}$) computes to

$$\ulcorner \underline{n+1} \urcorner = (\text{abs } (\text{abs } (\text{app } (\text{ref } \underline{1}) \ulcorner \underline{n} \urcorner)))$$

- the core of our proof is within the existence of a function $\text{hnf} \in \overline{\text{Nat}} \rightarrow \overline{\text{Nat}}$ s.t. for any $M \in \overline{\text{Nat}^p} \rightarrow \overline{\text{Nat}}$

$$(\text{hnf } \ulcorner M \urcorner) \downarrow \iff M \text{ has a HNF } N$$

and, in that case $(\text{hnf } \ulcorner M \urcorner) \rightarrow_h \ulcorner N \urcorner$. $\square$

5 Representation of algebraic data types

As far as possible all the previous results can be generalized to a large class of propositions. We give now a sketchy presentation of the way usual data structures such as lists, binary trees ... are dealt with in CC^+ .

5.1 Algebraic types

We begin with some notations and remarks. In that section polymorphic products and sums are defined as:

- **product** $A_1 \times \dots \times A_r$ stands for

$$(C : \overline{\text{Prop}})(A_1 \rightarrow \dots \rightarrow A_r \rightarrow C) \rightarrow C$$

- **sum** $A_1 + \dots + A_r$ for

$$(C : \overline{\text{Prop}})(A_1 \rightarrow C) \dots (A_r \rightarrow C) \rightarrow C$$

If $r = 0$ the two of them reduce to $(C : \overline{\text{Prop}})C$.

Note that the predicate $\text{Pos}_X(\cdot)$ is stable through these operations. In order to get a well-founded definition we need the following formalism [15]:

5.1.1 Definition Let $\Delta, X : \overline{\text{Prop}}$ a well-formed environment and $\mathfrak{P}$ a class of well-formed propositions on Δ .

(i) an **algebraic specification** of X on $\mathfrak{P}$ is a cartesian product $A_1 \times \dots \times A_s$ with $s \geq 0$ and

$$(\forall i) A_i \neq X \implies A_i \in \mathfrak{P}$$

(ii) for any list $\mathcal{L} = (F_i)_{1 \leq i \leq n}$ of algebraic specifications with $n \geq 0$, we define the **algebraic composition** of $\mathcal{L}$ to be

$$\oplus \mathcal{L} \equiv (X : \overline{\text{Prop}})F_1 + \dots + F_n \in \overline{\text{Prop}}$$

5.1.2 Definition The class of the **algebraic types** is the least class $\mathfrak{P}$ of well-formed propositions in the empty environment closed by algebraic composition.

5.1.3 Examples (i) for $\mathcal{L} = \emptyset$ we get $\oplus \mathcal{L} = {}_+ \perp \in \overline{\text{Prop}}$

(ii) the previous type Nat is given by $\mathcal{L} = \{\emptyset, X\}$

(iii) the type List of integers is given by $\{\emptyset, \text{Nat} \times X\}$

(iv) For $\{\emptyset, X \times X\}$ we get the binary trees. Let $\text{Bin} \equiv \oplus \mathcal{L}$. then

$$\text{Bin} = {}_+ (C : \overline{\text{Prop}})C \rightarrow (\text{Bin} \times \text{Bin} \rightarrow C) \rightarrow C$$

In the following lines an algebraic type R is fixed where $R \equiv \oplus \mathcal{L}$, with $\mathcal{L} = \{\emptyset, F_1, \dots, F_n\}$ is a list of specifications. We let $F_i \equiv A_1 \times \dots \times A_{s_i}$, and $r_i = \#\{i \mid A_i \equiv X\}$ is the **arity** of F_i . For short any term $M[X/R]$ is written M' . Thus $R = {}_+ \emptyset + F'_1 + \dots + F'_n$. For sake of simplicity we take as a notation that $A_1 \times \dots \times A_s \rightarrow B$ is identified with $A_1 \rightarrow \dots \rightarrow A_s \rightarrow B$.

With an algebraic type come some constructors and destructors defined as follow:

5.1.4 Definition (i) For each k the (closed) term

$$\begin{aligned} c_k &\equiv [\pi : F'_k][C : \overline{\text{Prop}}][f_0 : C][f_1 : F'_1 \rightarrow C] \\ &\quad \dots [f_n : F'_n \rightarrow C](f_k \pi) \\ &\in F'_k \rightarrow R \end{aligned}$$

is called a **constructor**.

(ii) we let **nil** (of R) to be $c_0 \in R$

(iii) for each k such that $r_k > 0$ the term

$$\begin{aligned} \pi_k^i &\equiv [x_1 : R] \dots [x_{r_k} : R][a_1 : A'_1] \dots [a_{p_k} : A'_{p_k}]x_i \\ &\in F'_k \rightarrow R \end{aligned}$$

is called a **destructor**.

(iv) for each k such that $r_k > 0$ we define the **predecessors according to F_k shape** (predecessors for short) as

$$\begin{aligned} \text{pred}_k^i &\equiv [r : R](r R \text{ nil } [p : F'_1] \text{ nil} \\ &\quad \dots \pi_k^i \dots [p : F'_n] \text{ nil}) \\ &\in R \rightarrow R \end{aligned}$$

5.1.5 Example For the binary trees we simply get

$\text{nil} \equiv [C : \overline{\text{Prop}}][Z : C][S : \text{Bin} \times \text{Bin} \rightarrow C]Z$
 $\text{cons} \equiv [a, b : \text{Bin}][C, Z, S](S \ a \ b)$
 $\text{left} \equiv [t : \text{Bin}](t \ \text{Bin} \ \text{nil} \ [a, b : \text{Bin}]a)$
 $\text{right} \equiv [t : \text{Bin}](t \ \text{Bin} \ \text{nil} \ [a, b : \text{Bin}]b)$

Anything we did on *Nat* can be done for *Bin* too. For instance we can define the class of (partial) computable functions which are definable in terms of standard elements of type $R^p \rightarrow R$ as the least class containing all constant functions, the constructors and destructors defined above and closed by composition, bar recursion and μ -recursion.

We refer to the book by H. Abelson and J. & G.J. Sussman for details. Actually almost all the programs given there can be typed within CC^+ . We may notice that the previous two kinds of recursion defined over type *Nat* illustrate the important fact that we get altogether terms as *lists* and terms as *streams*.

6 Some logical aspects

This section is intended to give a first insight on the interaction between partial and total terms. Our short overview makes use of the syntax.

6.1 #-terms

Let us introduce two kind constants : $\text{Prop}^\sharp$ and $\text{Type}^\sharp$ together with the same rules as for bar kinds $\overline{\text{Prop}}$ and $\overline{\text{Type}}$. Terms of these new kinds will be called #-terms.

Notice that strong normalisation for β -reductions is obviously preserved. So we may give the following results for terms in β -normal form with no loss of generality.

6.1.1 Definition Let M be as in 2.2.3, a #-term.

- (i) M is a #HNF if M' is not a redex (i.e. not a β fix-redex).
- (ii) M is a #-sensitive normal form, #SNF for short, if it is a #HNF and for all R_i which is a #-term, R_i is a #SNF.

Clearly this definition generalizes that of terms in ANF. But we needed no such generality in the previous sections.

6.2 Interaction between partial and total terms

We note CC_{tot}^+ , CC_{par}^+ and $CC_\sharp^+$ for the collection of total, partial and #-terms respectively.

6.2.1 Proposition There exists $(\cdot)^\sharp : CC_{\text{tot}}^+ \rightarrow CC_\sharp^+$ and $(\cdot)^\flat : CC_\sharp^+ \rightarrow CC_{\text{tot}}^+$ s.t. $(\forall M \in CC_{\text{tot}}^+) M^\sharp$ is a #SNF and $(M^\sharp)^\flat =_\alpha M$.

For example any valid judgment in CC gives rise to a #SNF since CC is embeddable in CC_{tot}^+ .

In fact we get more generality through the two next propositions.

6.2.2 Proposition (lifting) If $\Delta \vdash M \in N$ in CC_{tot}^+ then $\Delta^\sharp \vdash M^\sharp \in N^\sharp$.

6.2.3 Proposition (projection) Let $\Delta \vdash N \in \mathcal{K}$ within *Prop* or *Type*. If there exists $\Delta^\sharp \vdash M' \in N^\sharp$ in $CC_\sharp^+$ with M' in #SNF, then $\Delta \vdash M^b \in N$ is a valid judgment in CC_{tot}^+ .

In fact we will use to identify #-terms with partial terms. As an example of these results we get proofs for the predicate *Ind* applied to ANF terms of type *Nat* (see figure 4). Hence $\pi \in (x : \text{Nat})(\overline{\text{Ind}} \ x)$ can be seen as a *generic proof*.

6.2.4 Remark All the previous results can be given in a topological setting. Therefore terms in ANF are to be understood as compact terms for the corresponding topology. Since we restrict ourselves to syntactic tools this interpretation is studied in [2].

7 Conclusion

Introduction of fixpoints really enriches the Calculus of Constructions. As far as p.r. functions are concerned we get all the power of type-free calculus with the advantages of type checking. Computations, like that of the predecessor of type $\text{Nat} \rightarrow \text{Nat}$, are made more efficient.

Using algebraic types we get an internal representation of the terms and of inference rules of $CC^{(+)}$. We have also an internal representation of terms evaluation within the system. Thus we can dovetail computations in CC^+ and a parallel *or* can be built. Nonetheless strong normalisation is preserved as far as possible. We preserve the semi-decidability of the well-formedness of a term and of the equality of two terms. Following [14] we can extend easily system $F\omega$ and define an extraction function for informative terms. This lead us to a generalized predicate of realizability for our system. This could give us a good basis of comparison with inductive definitions related by D. Leivant [11]

This increase of power from the computational point of view is not yet followed by a clear insight of what the kind *Prop* means. So the logical aspects of the

extension need more examination. We hope that the construction of an appropriate model for CC^+ could help for that purpose. This is an open problem too.

Another direction of research concerns the comparison of our approach with the notions of generalized recursion theory developed by Moschovakis [12]. It is worth noticing that our definition of algebraic types restricts to 0-level types: taking functions as arguments is not allowed.

Acknowledgements

I am very pleased to thank Serge Yoccoz for his idea of extending CC with fixpoints. This work was done under his patient supervision. And many things emerged in our discussions. Actually I would have never submitted this paper if he did not encourage me to do so.

References

- [1] Abelson H., Sussman G.J., Sussman J. (1985) Structure and Interpretation of Computer Programs. The MIT Press (1985).
- [2] Audebaud Ph. (1991) Rapport Interne LaBRI (in preparation).
- [3] Barendregt H.P. (1984) *The Lambda-calculus: its syntax and semantics*. 2nd ed. North-Holland. Amsterdam.
- [4] Breazu-Tannen V. and Gallier J.H. (1989) Polymorphic rewriting conserves algebraic strong normalisation and confluence. *Proceedings of 16th International Colloquium on Automata, Languages and Programming* (ed. G. Ausello, M. Dezani-Ciancaglini and S. Ronchi della Rocca) Lectures Notes in Comp. Science **372**. Springer Verlag. Berlin.
- [5] Constable R.L. and Smith S.F. (1987) Partial objects in constructive type theory. In *2nd Conf. on Logic in Comp. Science*. IEEE, 1987.
- [6] Constable R.L. and Smith S.F. (1988) Computational foundations of basic recursive function theory. In *3rd Conf. on Logic in Comp. Science*. IEEE, 1988.
- [7] Coquand T. (1987) Metamathematical investigations of a Calculus of Constructions. Part I: Syntax. Unpublished ?
- [8] Coquand T. (1990) Metamathematical investigations of a Calculus of Constructions. In *Logic and Computer science*. P. Odifreddi editor. Academic Press Limited. London.
- [9] Coquand T. and Huet G. (1988) The Calculus of Constructions. *Inf. Comp.* **76** p.95-120.
- [10] Gallier J.H. (1990) On Girard's "Candidats de Réductibilité". In *Logic and Computer science*. P. Odifreddi editor. Academic Press Limited. London.
- [11] Leivant D. (1990) Contracting proofs to programs. In *Logic and Computer science*. P. Odifreddi editor. Academic Press Limited. London.
- [12] Moschovakis Y.N. (1983) Abstract Recursion as a foundation for Theory of Algorithms. In *Computation and Proof Theory*. Proceedings of Logic Colloquium, Aachen, 1983 - Part II. Lecture Notes in Mathematics **1104**. Springer Verlag. Berlin.
- [13] Parigot M. (1988) Programming with proofs: a second order type theory. *Proceedings of ESOP '88* (ed. H. Ganziger) Lectures Notes in Comp. Science **300**. Springer Verlag. Berlin.
- [14] Paulin-Mohring Ch. (1989) Extracting $F\omega$ programs from proofs in the Calculus of Constructions. In *Proceedings of 16th ACM Symposium on Principles of Programming Languages*. ACM, New-York.
- [15] Paulin-Mohring Ch. (1989) Inductive definitions in the Calculus of Constructions. (draft) In *The Calculus of Constructions* Rapport technique INRIA **110**.
- [16] Smith S.F. (1988) Partial Objects in Type Theory. Ph.D. Dissertation. Cornell University.

- if M is a proof then $\llbracket M \rrbracket_\rho = \sup(\varphi(M))$
- else case M :

$$\begin{aligned}
\llbracket c \rrbracket_\rho &= \sup(\varphi(c)) \\
\llbracket x \rrbracket_\rho &= \rho(x) \\
\llbracket (M_1 \ M_2) \rrbracket_\rho &= \llbracket M_1 \rrbracket_\rho \cdot \llbracket M_2 \rrbracket_\rho \\
\llbracket (x : P) M_1 \rrbracket_\rho &= \text{Fix}(\llbracket [x : P] M_1 \rrbracket_\rho) \\
\llbracket [x : P] M_1 \rrbracket_\rho &= \langle \varphi([x : P] M_1), \lambda \alpha \in \langle \varphi(P) \rangle \cdot \llbracket M_1 \rrbracket_{\rho[x \leftarrow \alpha]} \rangle \\
\llbracket (x : P) Q \rrbracket_\rho &= \langle \varphi((x : P) Q), \{ \mu \in \langle \varphi((x : P) Q) \rangle \mid (\forall \alpha \in \llbracket P \rrbracket_\rho) \mu \cdot \alpha \in \llbracket Q \rrbracket_{\rho[x \leftarrow \alpha]} \} \rangle
\end{aligned}$$

Figure 2: Interpretation of a term M

Let $F \equiv [x : \text{Nat}](x+1 = 0) \equiv [x : \text{Nat}](P : \text{Nat} \rightarrow \overline{\text{Prop}})(P \ x+1) \rightarrow (P \ 0)$ and $\perp \equiv (C : \overline{\text{Prop}})C$.

$$\begin{aligned}
\gamma &\equiv [h : (F \ 0)][P : \text{Nat} \rightarrow \overline{\text{Prop}}][g : (P \ 2)](h \ P \ (h \ [y : \text{Nat}](P \ y+1) \ g)) \\
&\in (F \ 0) \rightarrow (F \ 1) \\
\delta &\equiv [h : (F \ 0)][f : (F \ 1 \rightarrow \perp)](f \ (\gamma \ h)) \\
&\in (F \ 0) \rightarrow ((F \ 1) \rightarrow \perp) \rightarrow \perp \\
\pi &\equiv [h : (F \ 0)](h \ [x : \text{Nat}]((F \ x) \rightarrow \perp) \rightarrow \perp) \delta \ \pi) \\
&\in (F \ 0) \rightarrow \perp
\end{aligned}$$

Figure 3: A proof for $\neg(1 = 0)$

- case of Cnat

$$\begin{aligned}
Cind &\equiv [n : \text{Cnat}](P : \text{Cnat} \rightarrow \text{Prop})(P \ 0) \rightarrow ((k : \text{Cnat})(P \ k) \rightarrow (P \ (S \ k))) \rightarrow (P \ n) \\
&\in \text{Cnat} \rightarrow \text{Prop} \\
&\text{neither provable nor realizable}
\end{aligned} \tag{1}$$

- case of Nat

$$\begin{aligned}
\overline{Ind} &\equiv [n : \text{Nat}](\overline{P} : \text{Nat} \rightarrow \overline{\text{Prop}})(\overline{P} \ 0) \rightarrow ((k : \text{Nat})(\overline{P} \ k) \rightarrow (\overline{P} \ k+1)) \rightarrow (\overline{P} \ n) \\
&\in \text{Nat} \rightarrow \overline{\text{Prop}}
\end{aligned} \tag{2}$$

$$\begin{aligned}
\pi &\equiv \langle W : (n : \text{Nat})(\overline{Ind} \ n) \rangle [n : \text{Nat}][P, h, i](n \ (P \ n) \ h \ [k](i \ k \ (W \ k \ P \ h \ i))) \\
&\in (n : \text{Nat})(\overline{Ind} \ n)
\end{aligned} \tag{3}$$

$$\begin{aligned}
Ind &\equiv [n : \text{Nat}](P : \text{Nat} \rightarrow \text{Prop})(P \ 0) \rightarrow ((k : \text{Nat})(P \ k) \rightarrow (P \ k+1)) \rightarrow (P \ n) \\
&\in \text{Nat} \rightarrow \text{Prop}
\end{aligned} \tag{4}$$

$$\begin{aligned}
\alpha_0 &\equiv [P : \text{Nat} \rightarrow \text{Prop}][h : (P \ 0)][i : (k : \text{Nat})(P \ k) \rightarrow (P \ k+1)]h \\
&\in (Ind \ 0)
\end{aligned} \tag{5}$$

$$\begin{aligned}
\alpha_{n+1} &\equiv [P, h, i](i \ n \ (i \ n-1 \ \dots (i \ 0 \ h) \ \dots)) \\
&\in (Ind \ n+1)
\end{aligned} \tag{6}$$

Figure 4: Some predicates over integers